SOCKET PROGRAMMING

CIS*3210
Fall 2026

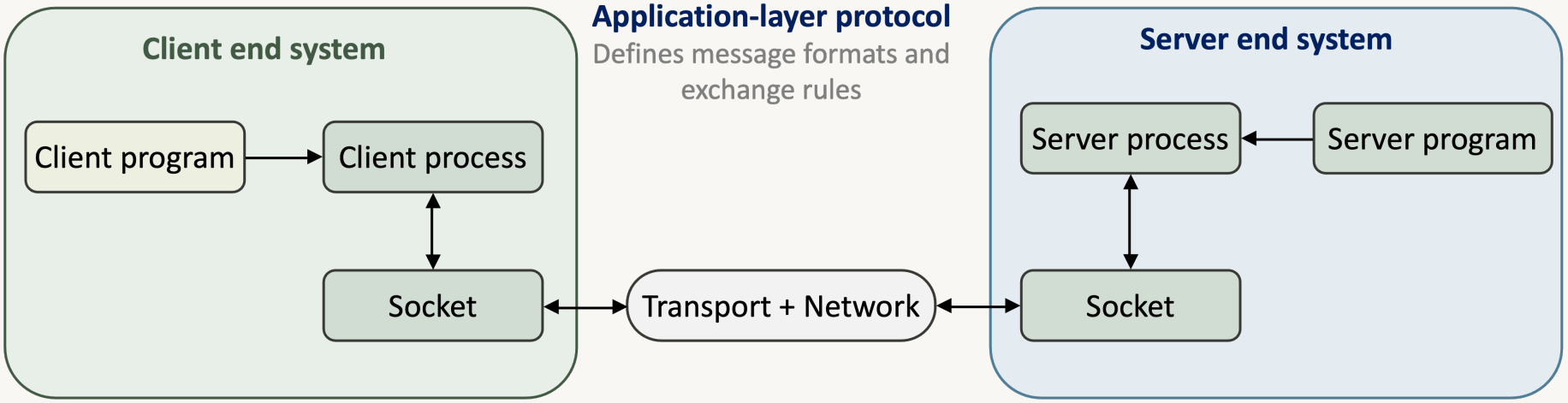
Sajid Marhon

Based on notes for Computer Networking: A Top-Down Approach, 9th ed., by J. Kurose and K. Ross

Socket Programming



- A network app is built as two programs, client and server, running on different end systems
  - when executed, they become processes that communicate by reading/writing through sockets
- The developer's core job is to implement both the client and the server code
- Two kinds of network applications:
  - **Open/standardized:** Follow a public protocol spec (e.g. RFC) Independent implementations interoperate if they follow the spec (e.g., Chrome ⟺ Apache; BitTorrent client ⟺ tracker)
  - **Proprietary:** Use a private, unpublished application-layer protocol. One team controls both ends; outside developers can't interoperate



Choosing a Transport Protocol and Port Number

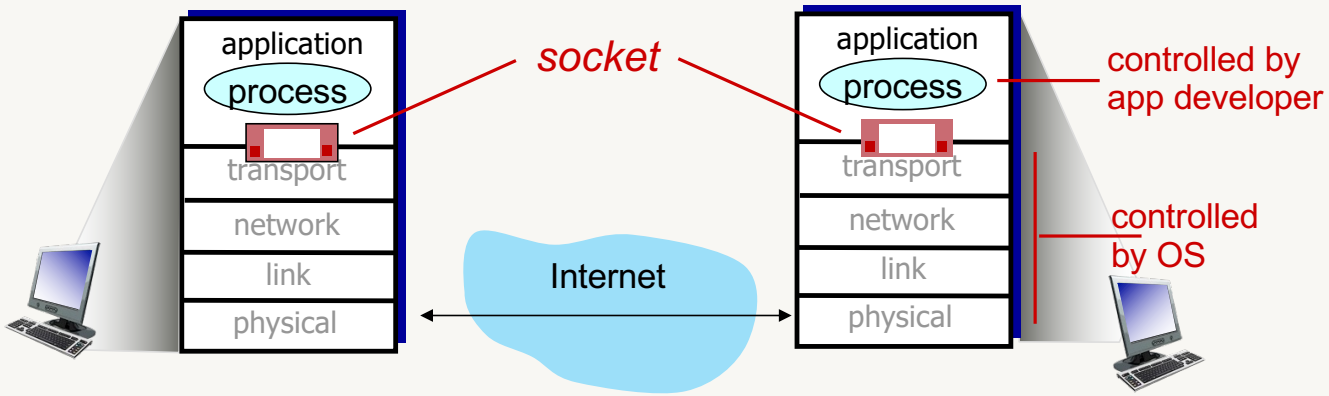


- Early design choice: which transport to use:
  - **TCP:** Connection-oriented, reliable byte-stream channel
  - **UDP:** Connectionless, sends independent packets, no delivery guarantees
- **Port numbers**:
  - If implementing an RFC-defined protocol, use its assigned well-known port
  - For proprietary apps, avoid well-known ports

Sockets: The Interface Between Applications and Transport



- A socket is the interface between an application process and the end-to-end transport layer
- Think of a socket as the gateway connecting an app process to the transport protocol
  - Endpoint through which an application talks to the transport protocol
  - Communication bridge between an app and the end-to-end transport service
  - Connection point linking an application process with TCP/UDP

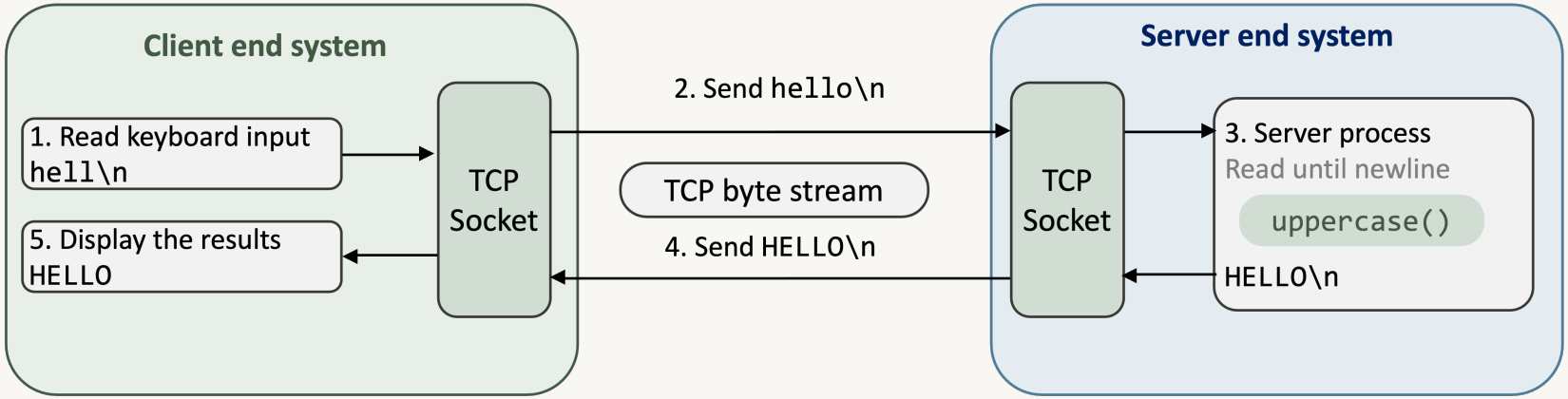


TCP and UDP Sockets: A Client–Server Example



- Two socket types map to two transport services:
  - **UDP:** best-effort, datagram-based
  - **TCP:** reliable, byte-stream–based
- Example application flow:
  - Client reads a line of text from the keyboard and sends it to the server
  - Server receives it and converts the characters to uppercase
  - Server sends the transformed text back
  - Client receives the result and displays the line

Example: convert a line of text to uppercase



Socket Programming With UDP



- Processes on different hosts exchange messages through **sockets** (app = inside the house; transport = outside; socket = door)
- **Developers** control the application side of a socket but have limited control over transport behavior
- With **UDP**, the sender specifies a destination **IP address and port** to reach the correct host and socket
- The source IP and port are also included, usually supplied by the **OS**
- **Demo app flow:**
  - Client reads a line from the keyboard and sends it
  - Server converts it to **uppercase** and sends it back
  - Client receives and displays the result
- The **server must be running** before the client sends a message

Socket Programming With UDP



- **Connectionless:** UDP doesn't establish a session between client and server, there's no setup/handshake
- **Per-packet addressing:** The sender must include the destination IP and port on every packet
- **Sender info from packet:** The receiver learns the sender's IP and port from the arriving packet
- **Best-effort delivery:** Packets can be lost, duplicated, or arrive out of order
- **App perspective:** UDP offers an unreliable, datagram-based transfer of byte groups between client and server processes

Client/Server Socket Interaction: UDP





server (running on serverIP)

create socket, port= x:

```
serverSocket = socket(AF_INET,SOCK_DGRAM)
serverSocket.bind(('', x))
```

read datagram from `serverSocket`

write reply to `serverSocket` specifying client address, port number

client 

create socket:

```
clientSocket =
socket(AF_INET,SOCK_DGRAM)
```

Create datagram with serverIP address and port=x; send datagram via `clientSocket`

read datagram from `clientSocket`

close `clientSocket`

Example: UDP Client



import Python's socket library

```
from socket import *
```

set server name/IP

```
serverName = 'hostname'
```

set server's socket port

```
serverPort = 12000
```

create UDP socket

```
clientSocket = socket(AF_INET,
    SOCK_DGRAM)
```

ask the client user to enter a sentence

```
message = input('Input lowercase
    sentence:')
```

convert message to byte type, attach the server name and port to it, send it to the process' socket

```
clientSocket.sendto(message.encode(),(se
    rverName, serverPort))
```

read reply data from socket

```
modifiedMessage, serverAddress =
    clientSocket.recvfrom(2048)
```

print the received message

```
print(modifiedMessage.decode())
```

close socket

```
clientSocket.close()
```

Example: UDP Server



```
from socket import *
serverPort = 12000
serverSocket = socket(AF_INET, SOCK_DGRAM)
serverSocket.bind(("", serverPort))
print("The server is ready to receive")
while True:
    message, clientAddress =
        serverSocket.recvfrom(2048)
    modifiedMessage = message.decode().upper()
    serverSocket.sendto(modifiedMessage.encode
    (), clientAddress)
```

create server socket

bind/assign a local port to server's socket

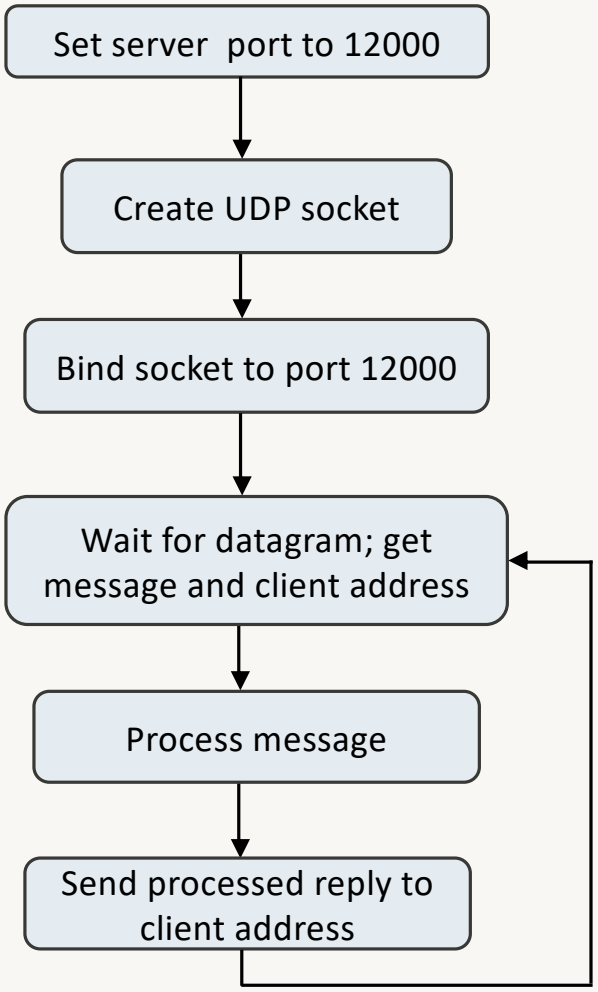
loop runs indefinitely

read message from UDP socket, get client's address (IP and port)

send uppercase string back to client

Example: UDP Server

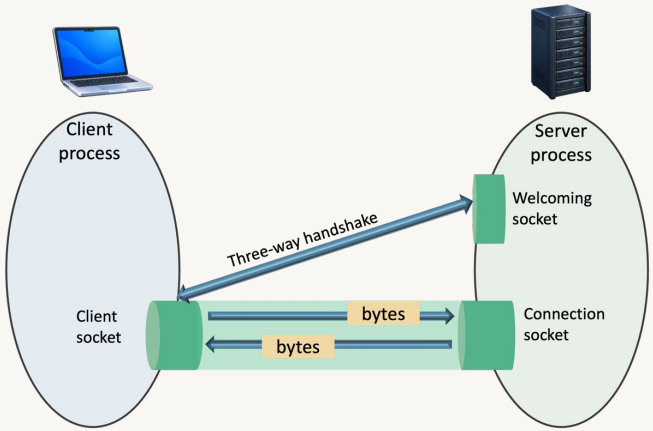




Socket Programming with TCP

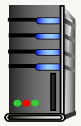


- **TCP is connection-oriented:** Before exchanging data, client and server complete an OS-managed **three-way handshake**
- **Endpoints:** A connection links the **client IP:port** and **server IP:port**, each associated with a socket
- **No address on each write:** Once connected, each side writes bytes to its socket without specifying the destination again
- **Server readiness:** The server must be running with a **listening socket** before clients connect
- **Connection setup:** The client creates a TCP socket and connects to the server IP and port; the server accepts it and creates a dedicated **connection socket**, separate from its listening socket
- **Service guarantees:** TCP provides a reliable, ordered, bidirectional byte stream between the connected sockets, much like a pipe



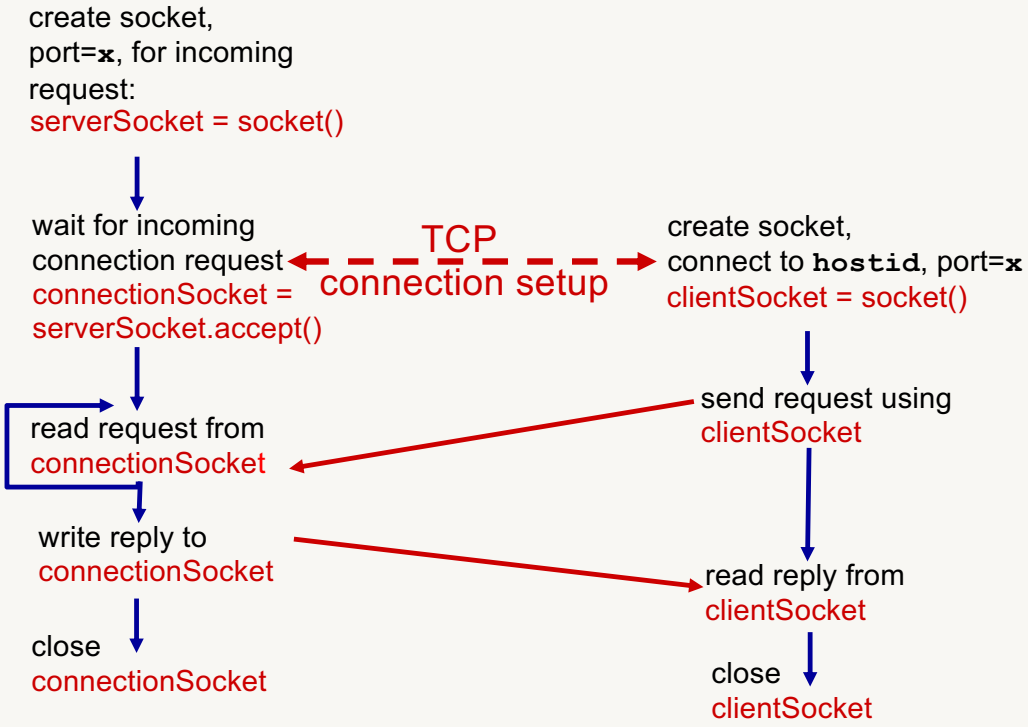
Client/Server Socket Interaction: TCP





server (running on hostid)

client 



Example: TCP Client



creates the TCP client's socket

establish a TCP connection btw client and server

send message thru client's socket to the TCP connection

store incoming bytes from the server

```
from socket import *

serverName = 'servername'

serverPort = 12000

clientSocket = socket(AF_INET,
    SOCK_STREAM)

clientSocket.connect((serverName,serverP
    ort))

sentence = input('Input lowercase
    sentence:')

clientSocket.send(sentence.encode())

modifiedSentence =
    clientSocket.recv(1024)

print('From Server: ',
    modifiedSentence.decode())

clientSocket.close()
```

Example: TCP Server



creates the TCP server's socket

server waits and listens to some client to knock the door

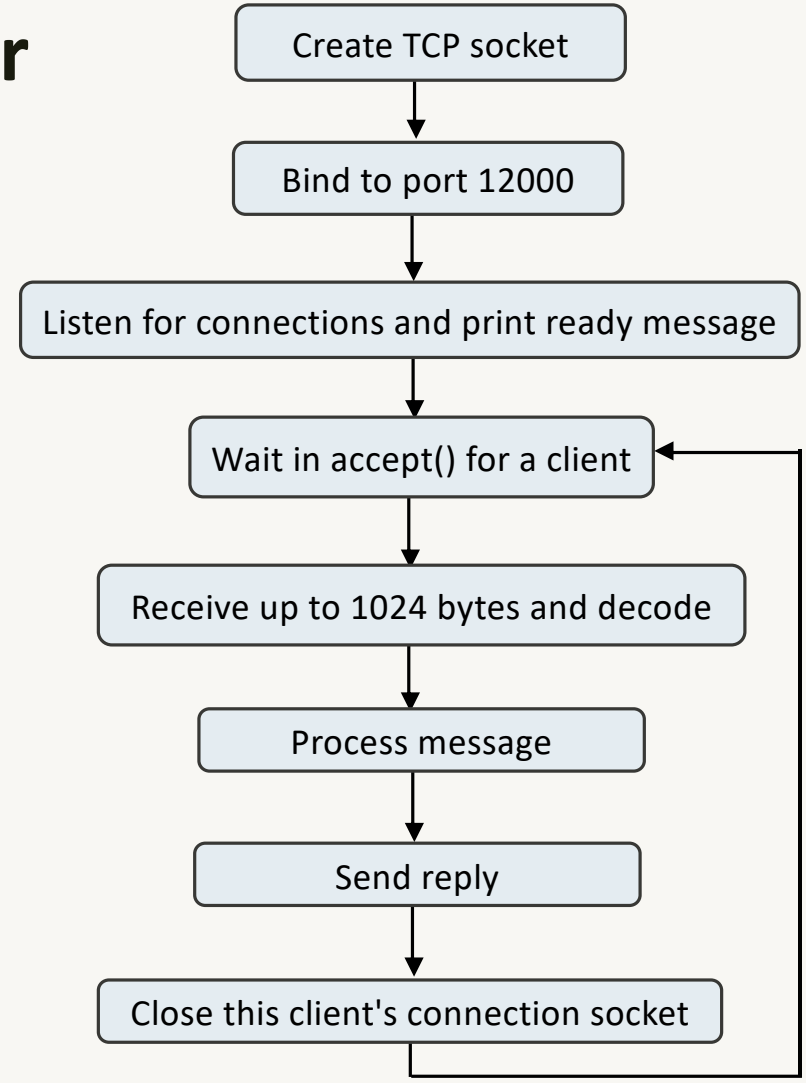
program invokes accept() method to create a new socket dedicated for this client

close connection to this client but not welcoming socket

```
from socket import *
serverPort = 12000
serverSocket = socket(AF_INET,SOCK_STREAM)
serverSocket.bind(('',serverPort))
serverSocket.listen(1)
print('The server is ready to receive')
while True:
    connectionSocket, addr =
    serverSocket.accept()
    sentence =
    connectionSocket.recv(1024).decode()
    capitalizedSentence = sentence.upper()
    connectionSocket.send(capitalizedSentence.
    encode())
    connectionSocket.close()
```

Example: TCP Server

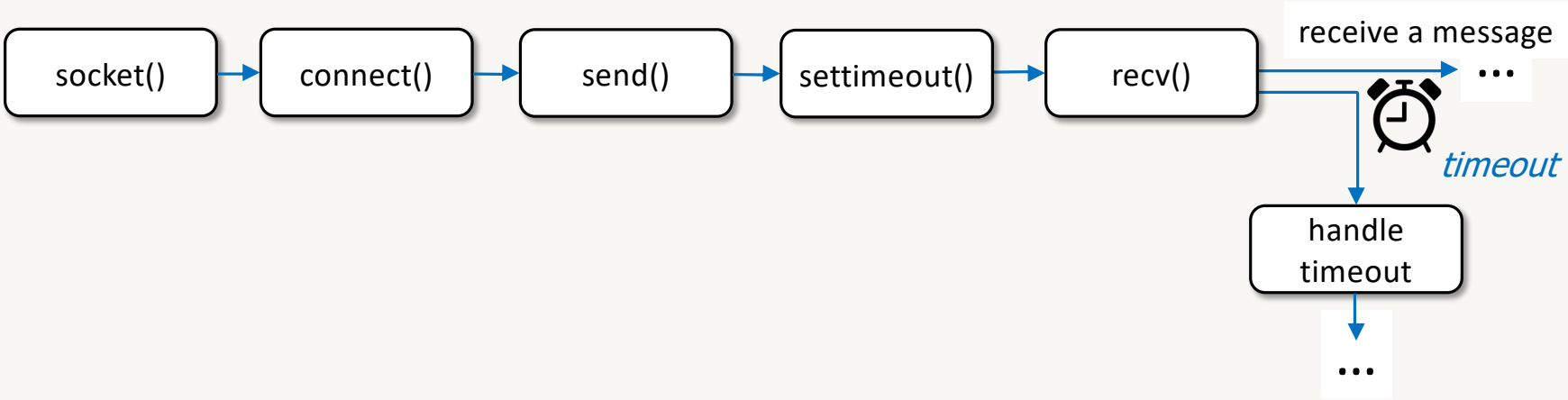




Waiting For Multiple Events

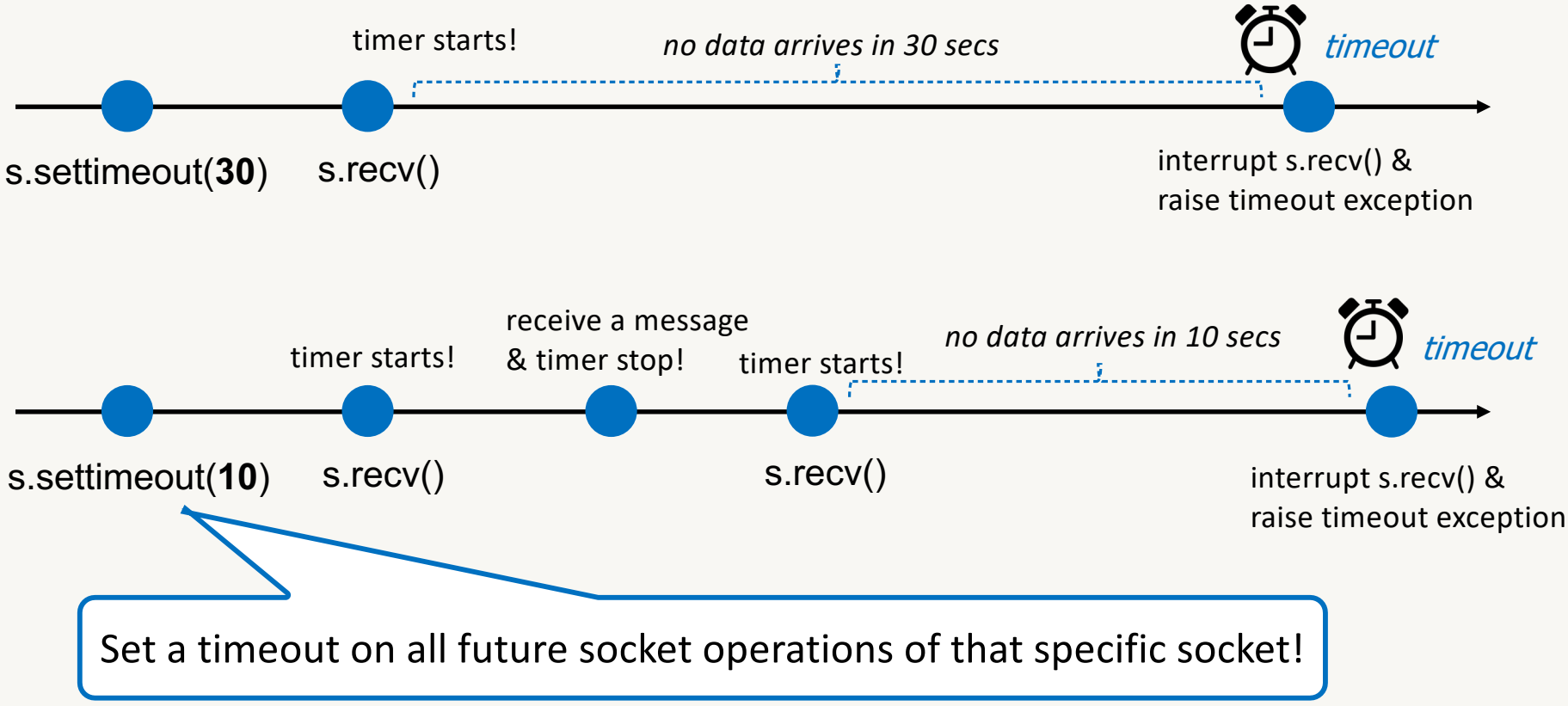


- Programs often need to wait on multiple possible events, e.g.
  - either a socket reply arrives, a timer expires,
  - or replies may come on several sockets (use select() or threads)
- **Timeouts** are widely used in networking
- In Python, set socket **timeouts** to handle these waits (socket.settimeout(...))



Python Socket Timeouts





Socket Programming with timeout



Example: Lab Sensor Alert (TCP)

- This example shows a minimal TCP server that listens on port 12000, accepts one client at a time, and waits up to 15 seconds for a single line of text from a lab sensor (e.g., “fridge-4, temp=9.2C, threshold=8.0C, Lab A”)
- When data arrives, the server calls a function notify_technician(...) to simulate paging the on-call tech, then sends back an acknowledgment: “technician notified”
- If no data arrives before the timeout, the server counts a timeout and continues;
  after four timeouts it stops

Socket Programming with Timeout



```
from socket import *
serverPort = 12000
serverSocket = socket(AF_INET,SOCK_STREAM)
serverSocket.bind(('',serverPort))
serverSocket.listen(1)
counter = 0
while counter < 4:
    connectionSocket, addr = serverSocket.accept()
    connectionSocket.settimeout(15)
    try:
        alert_line = connectionSocket.recv(1024).decode()
        notify_technician(alert_line)  # a lab function
        connectionSocket.send('technician
                notified'.encode())
    except timeout:
        counter += 1
connectionSocket.close()
```

set 15 sec timeout on all future socket operations

Time starts when recv() is called and will raise timeout exception if there is no message within 15 sec

catch socket timeout exception

References



- Ch. 2, Kurose and Ross



Questions?